

# CO5 ASSESSMENT 1

## CONSTRAINT-BASED PROBLEM SOLVING

### ANSWER DOCUMENT

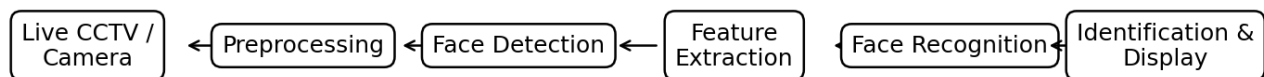
Question-wise Answers with OpenCV Pipeline Diagrams

#### 1. Real-Time Face Recognition for Campus Security



*Figure: Real-time face recognition using CCTV surveillance.*

#### OpenCV-Based Solution / Answer



*Figure: Relevant illustration for this OpenCV-based system.*

## **Answer**

A complete OpenCV-based face recognition system can be organized as a real-time pipeline from camera input to final identity display.

### **1. Image Acquisition**

Capture frames continuously using `cv2.VideoCapture()`. Resize frames to a suitable resolution so that processing remains fast on a low-cost computer. Skip unnecessary frames when required.

### **2. Preprocessing**

Convert the frame to grayscale or an appropriate color space, normalize illumination, and apply histogram equalization or CLAHE. Use light denoising and resize detected face regions to a fixed size. This improves consistency under different lighting conditions.

### **3. Face Detection**

Use a lightweight detector such as OpenCV Haar Cascade for low-cost hardware, or an optimized DNN face detector for better accuracy. The detector returns face bounding boxes, which are used to crop the face regions.

### **4. Feature Extraction**

Extract discriminative facial features from each aligned face. For a lightweight system, LBPH features can be used. For higher accuracy, a pretrained deep face embedding model can generate a compact numerical feature vector.

### **5. Recognition**

Compare the extracted feature or embedding with the registered student/staff database. Use a distance or confidence threshold. If the best match is below the threshold, display the registered identity; otherwise mark it as Unknown.

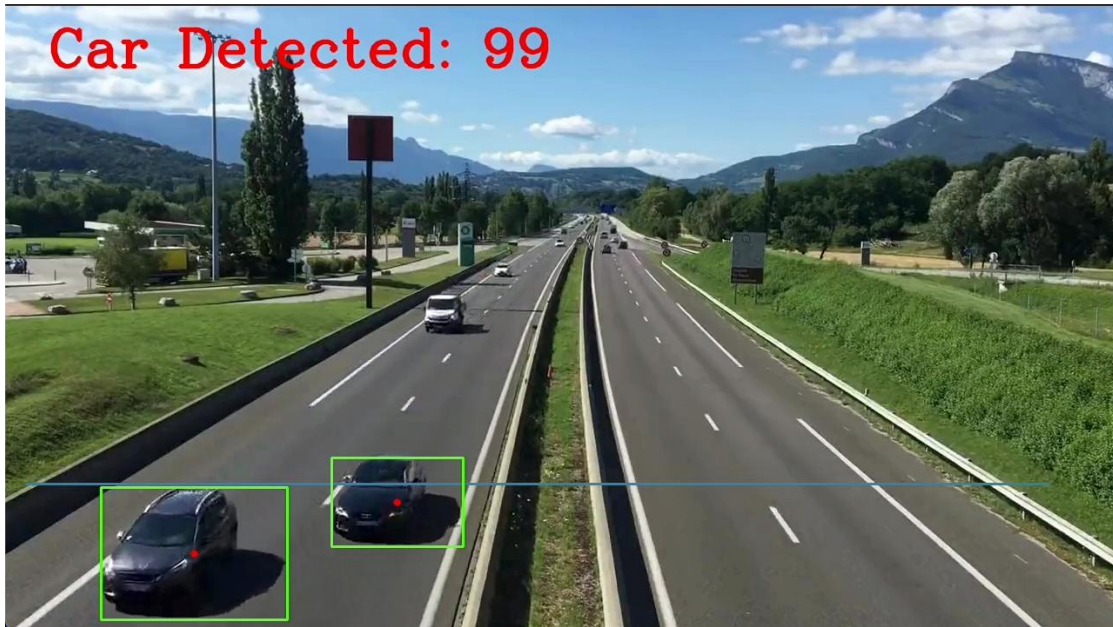
### **6. Optimization for Real Time**

Use reduced frame resolution, detect faces every few frames, track faces between detections, process only face regions, and use efficient OpenCV operations. Store registered features in advance instead of recalculating them.

### **Accuracy and Real-Time Justification**

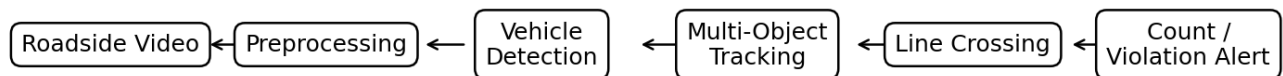
Lighting normalization and face alignment improve input quality; robust feature matching reduces false recognition; confidence thresholds reject uncertain matches. Resolution control, frame skipping, ROI processing, and tracking reduce computation, allowing continuous operation on a low-cost computer.

## 2. Intelligent Traffic Violation Detection System



*Figure: Relevant illustration for this OpenCV-based system.*

### OpenCV-Based Solution / Answer



## **Answer**

The system processes live roadside video and combines detection, tracking, line-crossing logic, and real-time visualization.

### **1. Video Acquisition and Preprocessing**

Read frames using OpenCV. Resize frames and apply light denoising. Convert to grayscale when motion analysis is required, while retaining color frames for visualization.

### **2. Vehicle Detection**

Use an OpenCV-compatible DNN detector to locate vehicles such as cars, buses, and motorcycles. Each detection is represented by a bounding box and confidence score.

### **3. Motion Analysis and Tracking**

Assign each vehicle a persistent ID using centroid tracking or a multi-object tracker. Match detections between consecutive frames using position and motion information.

### **4. Line-Crossing and Counting**

Define a virtual line or region of interest. When the tracked center of a vehicle moves from one side of the line to the other, increase the count only once for that vehicle ID.

### **5. Abnormal Movement Detection**

Analyze trajectory, direction, speed changes, and entry into restricted regions. Generate an alert for unwanted direction, abnormal movement, or other configured violations.

### **6. Handling Practical Problems**

Reduce shadows using color-based checks or shadow-aware background processing. Apply Gaussian/median filtering for camera noise, adaptive background models for changing scenes, and morphological opening/closing to clean motion masks. Multi-object tracking and ID history prevent the same vehicle from being counted repeatedly.

### **Visualization**

Draw bounding boxes, vehicle IDs, count values, virtual lines, and warning messages directly on the output frame using OpenCV drawing functions.

### 3. Automated Document Processing System



*Figure: Relevant illustration for this OpenCV-based system.*

#### OpenCV-Based Solution / Answer



#### Answer

The document processing pipeline should improve the scanned image before sending clean text regions to an OCR engine.

##### 1. Grayscale Conversion

Convert the input image to grayscale to reduce computational complexity and focus processing on text intensity rather than color.

##### 2. Noise Removal and Filtering

Use median or Gaussian filtering to reduce scanner noise, small dots, and blur effects while preserving important text structures.

##### 3. Illumination Correction and Thresholding

Use adaptive thresholding when illumination is uneven. Otsu thresholding can be used for documents with a clearer foreground/background separation. This converts the page into a high-contrast representation suitable for OCR.

##### 4. Morphological Operations

Use opening to remove small unwanted objects and closing to repair broken characters or text strokes. Kernel size should be selected carefully to avoid merging unrelated letters.

##### 5. Geometric Correction

Detect page boundaries, perspective distortion, and text skew. Apply rotation for deskewing and perspective transformation when the page was captured at an angle.

##### 6. Text Segmentation

Use connected components, contours, or projection profiles to separate text blocks, lines, and word regions. Pass only meaningful text regions to OCR.

## **7. OCR and Batch Processing**

Save the cleaned regions in a suitable format and process documents sequentially or in parallel batches. Store extracted text and processing status for efficient handling of thousands of files.

### **OCR Accuracy Justification**

Each stage removes a different source of error: grayscale simplifies processing, filtering reduces noise, thresholding increases contrast, morphology repairs text, geometric correction aligns characters, and segmentation prevents background objects from being interpreted as text.

## 4. Real-Time Multi-Person Tracking and Attendance System

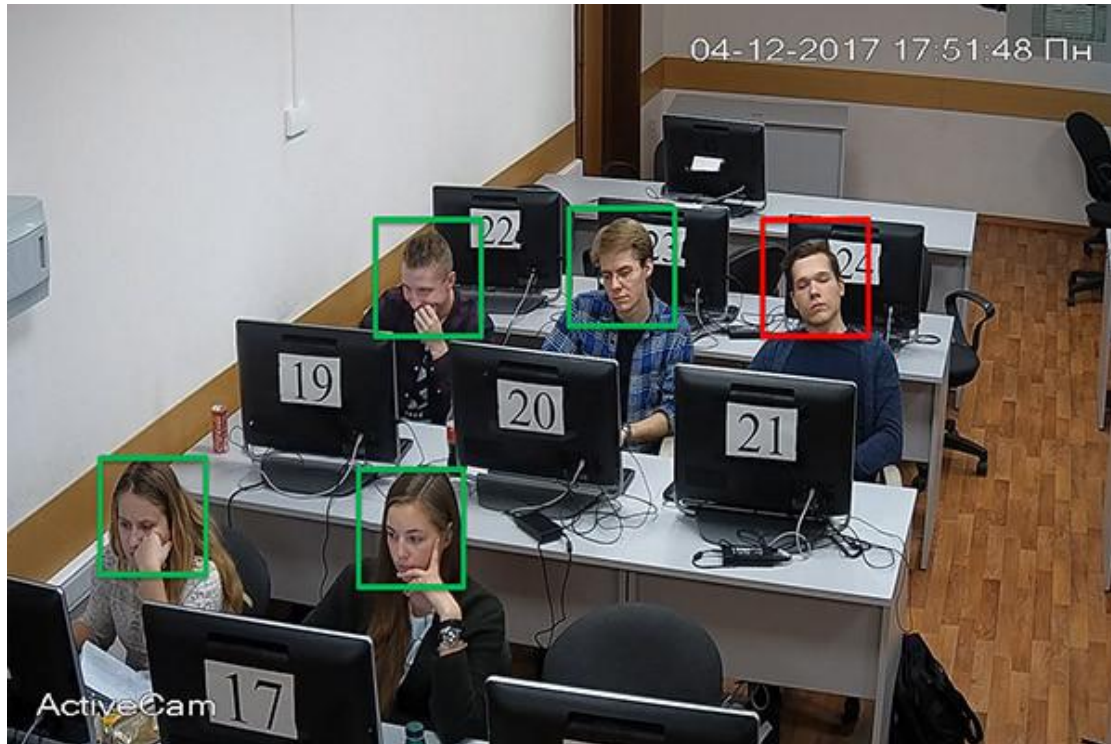
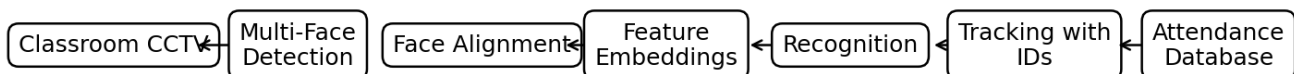


Figure: Relevant illustration for this OpenCV-based system.

### OpenCV-Based Solution / Answer



#### Answer

The attendance system combines multi-face detection, recognition, tracking, and a database rule that records each recognized student only once.

#### 1. Continuous Video Input

Capture classroom CCTV frames continuously and resize them for efficient processing.

#### 2. Multi-Face Detection

Detect all visible faces in a frame and generate a bounding box for each face. Use a detector capable of handling multiple simultaneous faces.

#### 3. Face Alignment and Feature Extraction

Normalize each detected face and extract a feature representation. Store feature vectors for registered students in advance.

#### **4. Recognition**

Compare each live feature vector with the registered database. Accept an identity only when the similarity/confidence satisfies a predefined threshold; otherwise keep it as Unknown.

#### **5. Identity Tracking Across Frames**

Assign a persistent track ID to each detected person. Use position overlap, motion prediction, or a tracking algorithm to associate the same person across consecutive frames.

#### **6. Handling Head Movement and Temporary Disappearance**

Allow tracks to remain active for a limited number of missed frames. Reassociate a returning face using its visual features and location, reducing unnecessary identity changes.

#### **7. Preventing Duplicate Attendance**

Maintain an attendance set or database keyed by student ID and date/session. Before marking attendance, check whether that student has already been recorded. Repeated appearances update the track but do not create another attendance entry.

#### **8. Reducing Delay and False Records**

Run expensive face detection periodically, use tracking between detections, process only detected face regions, and reject low-confidence recognition results. These measures reduce computation and false attendance.



## 5. Smart Home Surveillance and Intrusion Detection

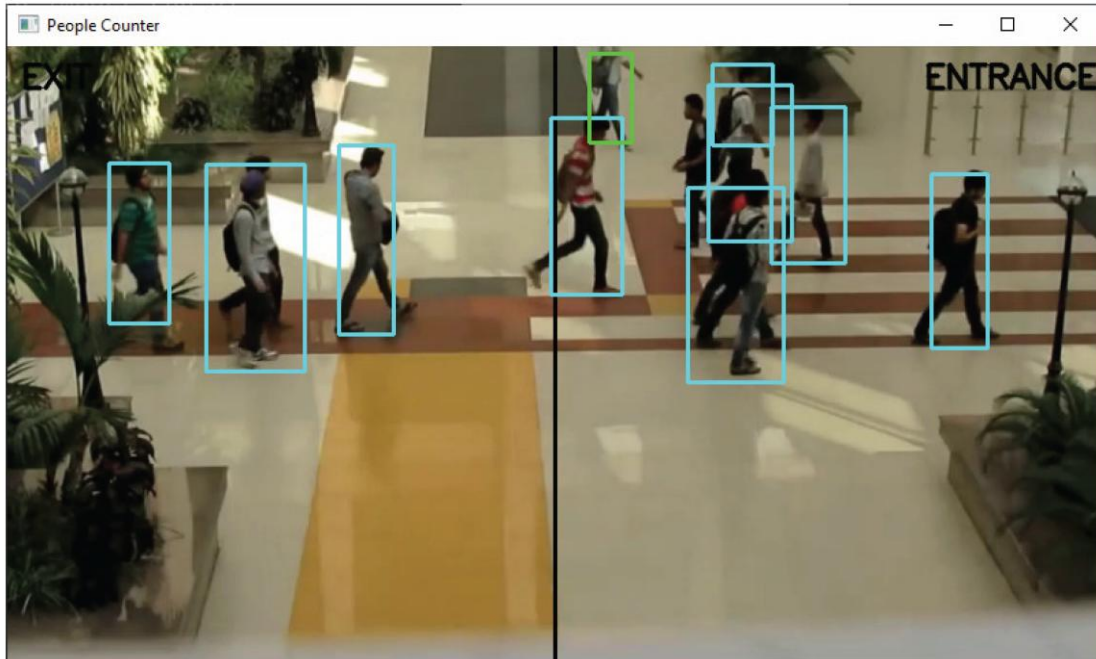
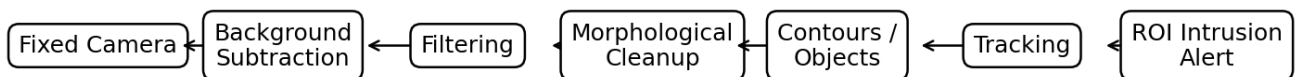


Figure: Relevant illustration for this OpenCV-based system.

### OpenCV-Based Solution / Answer



### Answer

A lightweight OpenCV surveillance pipeline can continuously model the background, isolate meaningful movement, track objects, and generate an alert only when configured intrusion conditions are satisfied.

#### 1. Background Subtraction

Use a background model such as MOG2 to separate moving foreground objects from the mostly static scene. This produces a foreground mask.

#### 2. Noise Filtering

Apply Gaussian or median filtering and remove very small regions that are likely to be camera noise.

#### 3. Morphological Operations

Use opening to remove isolated pixels and closing to fill gaps inside moving objects. This creates cleaner foreground blobs.

#### 4. Contour / Object Detection

Find contours in the cleaned mask and discard contours below a minimum area threshold. Remaining contours are represented using bounding boxes or centroids.

### **5. Motion Analysis and Tracking**

Track object position and movement across frames. Analyze whether the object is moving consistently, entering a protected area, or remaining in a suspicious location.

### **6. Unauthorized Region Entry**

Define a region of interest at the entrance. Trigger an alert only when a sufficiently large, persistent tracked object enters the protected region. If person identification is available, authorization can be checked before generating the final alert.

### **7. Reducing False Alarms**

Shadow handling and adaptive background updating reduce sensitivity to illumination changes. Area, persistence, direction, and region rules help ignore tree movement and small background changes. Filtering and morphology suppress camera noise.

### **Real-Time Operation**

Use a fixed resolution, process only the important region of interest, apply lightweight background subtraction, and track detected blobs rather than performing expensive analysis on every pixel or frame.